

Project 3

zkVM-guest API Standardization

6th Breakout Call · standards recap, a reworked draft, and vendor results

github.com/eth-act/zkvm-standards

Standards Recap

Accepted (navy)

In review (amber)

1 ACCEPTED

RISC-V Target

Minimal ISA riscv64im_zicclsm — one 64-bit target for every zkVM.

3 ACCEPTED

Cryptographic Accelerators

A C interface for the EVM precompile operations.

5 ACCEPTED

Memory Layout Restrictions

Each vendor owns its memory map; portability stays at the source level.

7 ACCEPTED

Instruction Address Misaligned

A misaligned jump target aborts. No rounding, no recovery.

9 IN REVIEW · #33

ELF Loading & Validation

How an ELF becomes the initial machine state. Detailed next.

2 ACCEPTED

IO Interface

Portable read_input / write_output for private input and public output.

4 ACCEPTED

Termination Semantics

Success may produce a proof; failure must not.

6 ACCEPTED

Memory Safety Guard Regions

Mandatory null-pointer trap and stack guard — violations abort.

8 ACCEPTED

Static Library & Linker Script

The vendor-supplied guest runtime. Accepted since Breakout 5.

10 IN REVIEW · #32

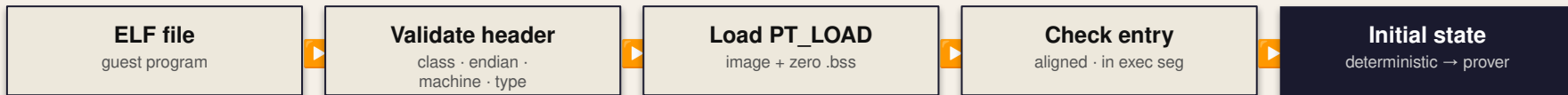
Accelerated Memory Operations

Vendor-accelerated memcpy / memmove / memset / memcmp.

Two standards are open for your review — please comment on the PRs.

In Review: ELF Loading & Validation

PR #33 · changed heavily since Breakout 5 — now defines what an executable segment means



New since Breakout 5 — an executable segment means one of two things:



A zkVM supports exactly one, as an inherent property. Recommended: accept PF_X | PF_R — no toolchain can be relied on to keep read-only data (e.g. jump tables) out of the code.

— The ELF is the authoritative initial state; the image comes from the PT_LOAD program headers — so stripped binaries still load.

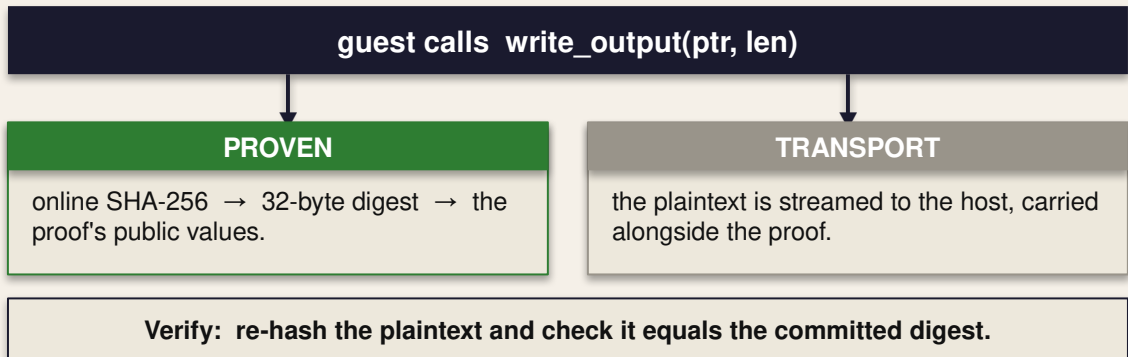
— Strict header checks and W^X reject an invalid ELF before any proof. e_flags is intentionally not checked — a zkVM may exceed the minimal target (for example, the compressed extension).

Addresses and region sizes stay vendor-defined — this fixes how an ELF is read, not where memory lives.

Vendor Result: Standard-Compliant Output on ZisK

0xPolygonHermes/zisk PR #1145 · a working implementation of the IO output standard

The IO standard mandates arbitrary-length public output — `write_output` can be called repeatedly, and the bytes concatenate.



Arbitrary-length output today

SP1 ✓ supported

ZisK + #1145 ✓ supported

other zkVMs ✗ not yet

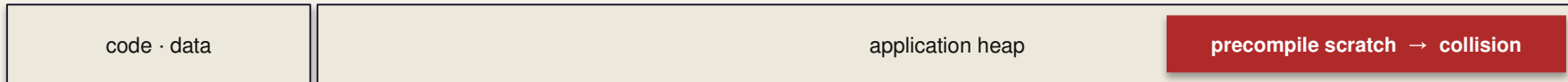
- Soundness rests only on the digest, so the transport can't forge output.
- Plaintext is detachable: stripped for recursion, re-attached for final delivery.

Vendor Result: Heap Ownership & Isolated Scratch

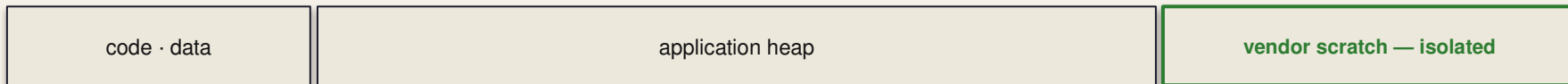
0xPolygonHermes/zisk PR #1157 · the standard gives the whole heap to the application

The standard: the heap belongs entirely to the guest application (`_heap_start` / `_heap_end`).

Before — vendor precompiles allocate inside the guest heap



After (PR #1157) — the library keeps its own isolated scratch; the heap is untouched



— So vendor precompiles and IO in the static library must never touch the guest's heap — they need their own memory.

— Fix: an isolated bump allocator, reset at every host → library call. The library is single-threaded, so memory stays bounded to one call.

Action for other vendors

Check whether the same heap collision affects your zkVM — a precompile or IO routine that quietly allocates inside the guest's heap is easy to miss.

Direction: No Modified Toolchains

a recommendation today · a candidate standard tomorrow

Ethereum guest programs should be built with a stock, unmodified toolchain.

Stock toolchain — recommended

Guest application

Standard compiler

trusted · widely tested · unchanged

small, well-understood trusted base

Modified toolchain

Guest application

Standard compiler

Vendor compiler patches ⚠

correctness of ALL emitted code now in question

the Trusted Computing Base grows

The trade-off we accept

A modified compiler could emit code tuned for zkVM execution. We give that up on purpose: a zkVM can absorb the ISA-level cost differences, and correctness is the priority. A smaller Trusted Computing Base is worth more than a faster guest.

Open for your review

ELF Loading & Validation (#33) **Accelerated Memory Operations (#32)**

Please comment on the PRs — and if you'd like engineering help implementing any standard, the Ethereum Foundation is glad to support.